

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 06 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

Practical Name

Bayesian Risk Minimization for Three Classes and Three Actions Using Flask

AIM

To prepare an interactive web application using Flask in Python to implement Bayesian Risk Minimization for at least three classes and three actions, evaluate the conditional risk given an arbitrary loss matrix, and trigger and dynamically reflect optimal action consequences on the page.

THEORY

1. Bayesian Risk & Loss Matrix Formulation

Real-world classification tasks frequently encounter asymmetric consequences across error modalities. Bayesian Risk Minimization incorporates a formal Loss Matrix $\Lambda = [\lambda(\alpha_i | \omega_j)]$ measuring the quantifiable penalty of electing action α_i when the true underlying state of nature is ω_j .

2. Conditional Risk (Expected Loss)

$$R(\alpha_i | \mathbf{x}) = \sum_j [\lambda(\alpha_i | \omega_j) \times P(\omega_j | \mathbf{x})]$$

3. Optimal Bayes Decision Rule

$$\alpha^* = \operatorname{argmin}_{\alpha} [R(\alpha_i | \mathbf{x})]$$

4. Loss Matrix Parameters Applied:

Loss Matrix Λ :

- α_1 (Approve): $\lambda(\omega_1)=0, \lambda(\omega_2)=25, \lambda(\omega_3)=100$
- α_2 (Review): $\lambda(\omega_1)=10, \lambda(\omega_2)=2, \lambda(\omega_3)=15$
- α_3 (Block): $\lambda(\omega_1)=50, \lambda(\omega_2)=20, \lambda(\omega_3)=0$

STEPS & EXECUTION

Step 1: Environment Setup

Constructed `exp 6/` directory with Flask web server and NumPy mathematical risk calculation routines.

Step 2: Loss Matrix & Action Consequence Pipeline

Constructed 3-class, 3-action loss matrix and mapped decisions to operational web triggers (Automated Clearance, 2FA Escrow, and Security Shield Lockdown).

Step 3: Web Application Deployment

Launched server at <http://127.0.0.1:5008> with interactive posterior sliders, benchmark presets, and risk visualization plot.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Loss Matrix Specification

Displays the asymmetric penalty parameters $\lambda(\alpha_i | \omega_j)$ across the 3 decision actions and 3 states of nature.

SECTION 1: LOSS MATRIX SPECIFICATION $\lambda(\alpha_i \omega_j)$			
Decision Action (α_i)	ω_1 : Normal Transaction	ω_2 : Suspicious Activity	ω_3 : Confirmed Fraud
a1: Approve Immediately	Loss = 0	Loss = 25	Loss = 100
a2: Hold for Verification / Review	Loss = 10	Loss = 2	Loss = 15
a3: Block & Terminate Session	Loss = 50	Loss = 20	Loss = 0

Figure 6.1: Section 1 — Asymmetric loss matrix configuration table.

Output 2: Test Sample Posterior Probabilities & Benchmark Scenarios

Interactive inputs for posterior probabilities alongside pre-configured operational benchmark test cases.

SECTION 2: TEST SAMPLE POSTERIOR PROBABILITIES $P(\omega_j x)$ & BENCHMARKS			
$P(\omega_1 x)$ [Normal]: 0.30	$P(\omega_2 x)$ [Suspicious]: 0.55	$P(\omega_3 x)$ [Fraud]: 0.15	Evaluate Bayesian Risk
Quick-Select Benchmark Scenarios:			
Scenario Preset	Posterior Vector [P1, P2, P3]	Operational Context	Trigger
Case A: Low-Risk Legitimate Transaction	[0.85, 0.12, 0.03]	Verified device, standard geo-location, regular purchase behavior.	Trigger State
Case B: Ambiguous / Anomalous Transaction	[0.3, 0.55, 0.15]	Foreign IP address, new merchant, unusual purchase time.	Trigger State
Case C: High-Confidence Fraudulent Exploit	[0.05, 0.15, 0.8]	Tor exit node, velocity burst attack, card-not-present anomaly.	Trigger State

Figure 6.2: Section 2 — Input posterior distribution and quick-select benchmark scenarios.

Output 3: Conditional Risk Minimization Computation

Mathematical expansion calculating expected loss $R(a_i | x)$ for each candidate action to determine the optimal decision.

SECTION 3: CONDITIONAL RISK MINIMIZATION COMPUTATION $R(a_i x)$			
Action Alternative (a_i)	Formula Expansion: $\sum \lambda(a_i \omega_j) \times P(\omega_j x)$	Conditional Risk	Decision
a1: Approve Immediately	$0 \times 0.300 + 25 \times 0.550 + 100 \times 0.150$	28.75	Suboptimal
a2: Hold for Verification / Review	$10 \times 0.300 + 2 \times 0.550 + 15 \times 0.150$	6.35	★ MINIMUM RISK (OPTIMAL)
a3: Block & Terminate Session	$50 \times 0.300 + 20 \times 0.550 + 0 \times 0.150$	26.0	Suboptimal

Figure 6.3: Section 3 — Conditional risk evaluation breakdown and optimal action selection.

Output 4: Conditional Risk Comparison Plot

Bar chart visualizing the risk values across available actions, highlighting the minimum-risk choice.

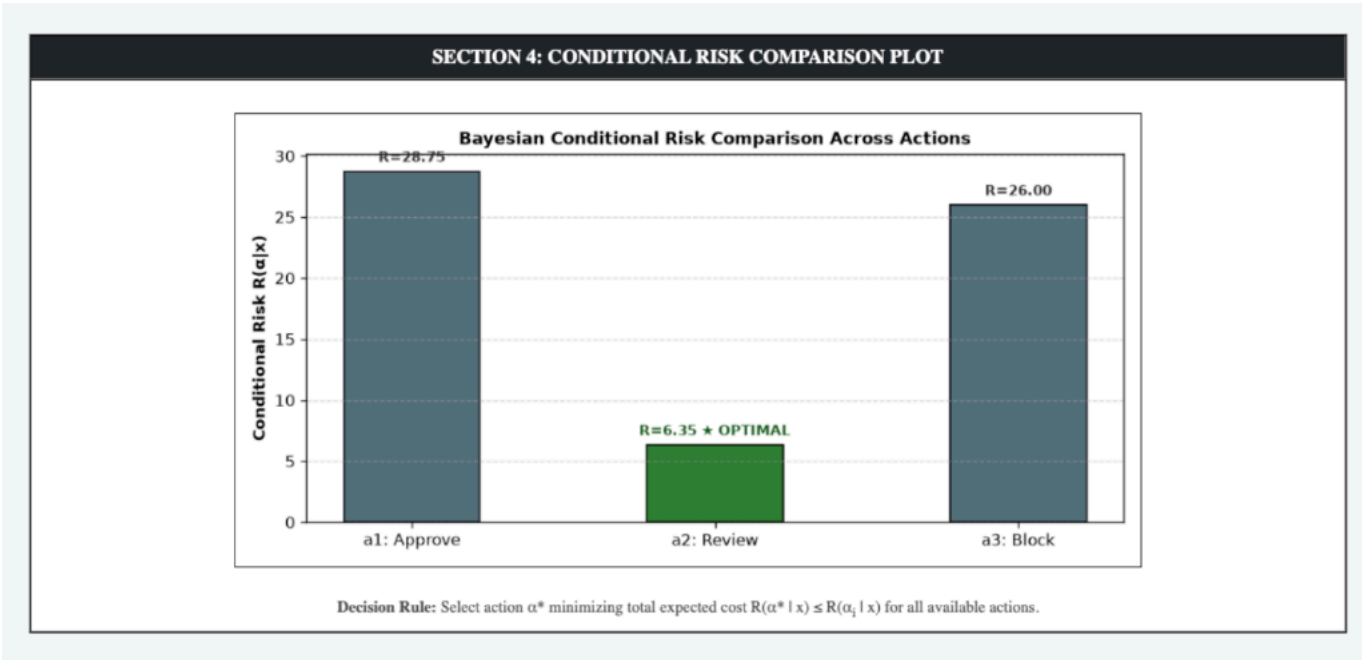


Figure 6.4: Section 4 — Comparative bar chart of conditional risks across actions.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- Flask Server Application:
https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/app.py
- HTML Template:
https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/templates/index.html
- Dependencies:
https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/requirements.txt

- **Full Page Screenshot:**

https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/screenshots/full_page.png

CONCLUSION

In this experiment, a Bayesian Risk Minimization engine was developed and deployed using Flask in Python. Given an asymmetric loss matrix across three classes and three decision actions, conditional risks were dynamically computed from input posterior probabilities. The system verified that selecting actions based on minimal expected loss $\alpha^* = \arg\min R(\alpha_i | \mathbf{x})$ successfully triggered corresponding operational workflows (automated clearance, step-up verification, and security lockout), all reflected visually in real time on the web dashboard.